

Configuring an AWS Site-to-Site VPN

To Begin with the Lab

Summary of the Lab

In this lab, an AWS Site-to-Site VPN was configured to securely connect an AWS VPC in the Mumbai region with a simulated on-premises data centre hosted as another VPC in the N. Virginia region. Private and public subnets, route tables, EC2 instances, a Virtual Private Gateway (VGW), Customer Gateway (CGW), and a Site-to-Site VPN connection using static routing were created. The VPN server was configured with LibreSwan using the downloaded AWS VPN configuration, and IP forwarding was enabled to establish the IPsec tunnel. Route tables were updated on both sides to direct traffic through the VPN, and Source/Destination Check was disabled on the VPN server so it could act as a router. Finally, connectivity was verified by successfully pinging and accessing private EC2 instances across the VPN tunnel, confirming secure communication between the two networks.

Note: In this lab, an AWS VPC is connected to a simulated on-premises data centre using an **AWS Site-to-Site VPN**. Since a physical data centre is not available, another VPC in a different AWS Region is used to simulate the on-premises network.

Create the AWS Network

- Open the **AWS Management Console** and switch to the **Mumbai (ap-south-1)** region. Create a VPC named **AWS-VPC** with the CIDR block **10.0.0.0/16**, which will represent the AWS cloud network for this VPN connection.

Create VPC [Info](#)

A VPC is an isolated portion of the AWS Cloud populated by AWS objects, such as Amazon EC2 instances.

VPC settings

Resources to create [Info](#)
Create only the VPC resource or the VPC and other networking resources.

☒ VPC only ☐ VPC and more

Name tag - optional
Creates a tag with a key of 'Name' and a value that you specify.

IPv4 CIDR block [Info](#)
☒ IPv4 CIDR manual input
☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR

CIDR block size must be between /16 and /28.

IPv6 CIDR block [Info](#)
☒ No IPv6 CIDR block
☐ IPAM-allocated IPv6 CIDR block
☐ Amazon-provided IPv6 CIDR block
☐ IPv6 CIDR owned by me

- Create a **Private Subnet** inside the AWS VPC using the CIDR block **10.0.0.0/24**. Since this subnet represents internal AWS resources, it does not require an Internet Gateway or public internet access.

Subnet settings
Specify the CIDR blocks and Availability Zone for the subnet.

Subnet 1 of 1

Subnet name
Create a tag with a key of 'Name' and a value that you specify.

The name can be up to 256 characters long.

Availability Zone Info
Choose the zone in which your subnet will reside, or let Amazon choose one for you.

IPv4 VPC CIDR block Info
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

IPv4 subnet CIDR block
 256 IPs

- Create a dedicated **Route Table** for the private subnet and associate it with the subnet. This route table will later be updated to send traffic destined for the on-premises network through the VPN Gateway.

Create route table Info
A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.

Route table settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

VPC
The VPC to use for this route table.

Tags
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key **Value - optional**

You can add 49 more tags.

- Associate the Subnet

Edit subnet associations
Change which subnets are associated with this route table.

Available subnets (1/1)

☒	Name	Subnet ID	IPv4 CIDR	IPv6 CIDR	Route table ID
☒	AWS-Private-Subnet	subnet-082fd5b90889f8b9c	10.0.0.0/24	-	Main (rtb-0aabe3c25ee50147)

Selected subnets

- Launch an EC2 instance named **EC2-A** inside the private subnet. Disable **Auto-assign Public IP** and configure the Security Group to allow **ICMP (Ping)** traffic from the on-premises network CIDR (**192.168.0.0/16**) for connectivity testing.
- Choose AMI as Amazon Linux

Launch an instance Info

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags Info

Name

AWS-A

Add additional tags

Application and OS Images (Amazon Machine Image) Info

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Search our full catalog including 1000s of application and OS images

Recents

My AMIs

Quick Start



Browse more AMIs
Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Summary

Number of instances Info

1

Software Image (AMI)

Amazon Linux 2023 AMI 2023.12....[read more](#)
ami-0d351f1b760a30161

Virtual server type (instance type)

t3.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free

Cancel

Launch instance

[Preview code](#)

- Choose VPC and private subnet that was created.

Network settings Info

VPC - *required* Info

vpc-03b0d69e546a3e416 (AWS-VPC)

10.0.0.0/16

Subnet Info

subnet-082fd5b90889f8b9c

VPC: vpc-03b0d69e546a3e416 Owner: 463646775279

Availability Zone: ap-south-1a (aps1-az1) Zone type: Availability Zone

IP addresses available: 251 CIDR: 10.0.0.0/24

AWS-Private-Subnet

Create new subnet

Auto-assign public IP Info

Disable

Firewall (security groups) Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

Create security group

Select existing security group

Security group name - *required*

aws-app-sg

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and ., -, /, #, @, [], +, =, &, ;, !, *, ~, `

Description - *required* Info

launch-wizard-1186 created 2026-07-01T08:14:27.604Z

Summary

Number of instances Info

1

Software Image (AMI)

Amazon Linux 2023 AMI 2023.12....[read more](#)
ami-0d351f1b760a30161

Virtual server type (instance type)

t3.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free

Cancel

Launch instance

[Preview code](#)

- Add inbound rule and click Launch Instance.

Inbound Security Group Rules

Security group rule 1 (TCP, 22, 0.0.0.0/0)

Remove

Type Info

ssh

Protocol Info

TCP

Port range Info

22

Source type Info

Anywhere

Source Info

Add CIDR, prefix list or security group

0.0.0.0/0

Description - *optional* Info

e.g. SSH for admin desktop

Security group rule 2 (ICMP, All, 192.168.0.0/16)

Remove

Type Info

Custom ICMP - IPv4

Protocol Info

All

Port range Info

All

Source type Info

Custom

Source Info

Add CIDR, prefix list or security group

192.168.0.0/16

Description - *optional* Info

e.g. SSH for admin desktop

Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

Add security group rule

Summary

Number of instances Info

1

Software Image (AMI)

Amazon Linux 2023 AMI 2023.12....[read more](#)
ami-0d351f1b760a30161

Virtual server type (instance type)

t3.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free

Cancel

Launch instance

[Preview code](#)

Create the Simulated Data Center Network

- Switch to the **US East (N. Virginia)** region and create another VPC named **DC-VPC** using the CIDR block **192.168.0.0/16**. This VPC represents the simulated on-premises data centre that will connect to AWS through the VPN.

Create VPC [Info](#)

A VPC is an isolated portion of the AWS Cloud populated by AWS objects, such as Amazon EC2 instances.

VPC settings

Resources to create [Info](#)
Create only the VPC resource or the VPC and other networking resources.

☒ VPC only ☐ VPC and more

Name tag - optional
Creates a tag with a key of 'Name' and a value that you specify.

DC-VPC

IPv4 CIDR block [Info](#)
☒ IPv4 CIDR manual input ☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR
192.168.0.0/16
CIDR block size must be between /16 and /28.

- Create an **Internet Gateway**, attach it to the Data Center VPC

[VPC](#) > [Internet gateways](#) > Attach to VPC (igw-0c73b9fd47aced980)

🔔 The following internet gateway was created: igw-0c73b9fd47aced980 - DC-IGW. You can now attach to a VPC to enable the VPC to communicate with the internet. [Attach to a VPC](#) ✕

Attach to VPC (igw-0c73b9fd47aced980) [Info](#)

VPC
Attach an internet gateway to a VPC to enable the VPC to communicate with the internet. Specify the VPC to attach below.

Available VPCs
Attach the internet gateway to this VPC.

Q vpc-0fc94dc2cfdab75d0 ✕

▶ AWS Command Line Interface command

[Cancel](#) [Attach internet gateway](#)

- Attach it to the VPC

Create internet gateway [Info](#)

An internet gateway is a virtual router that connects a VPC to the internet. To create a new internet gateway specify the name for the gateway below.

Internet gateway settings

Name tag
Creates a tag with a key of 'Name' and a value that you specify.

DC-IGW

Tags - optional
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional	
Q Name ✕	Q DC-IGW ✕	Remove

[Add new tag](#)
You can add 49 more tags.

[Cancel](#) [Create internet gateway](#)

- Create two subnets: a **Public Subnet** (192.168.0.0/24) and a **Private Subnet** (192.168.100.0/24). The public subnet will host the VPN Server, while the private subnet will contain the internal application server
- Public Subnet:

Subnet 1 of 1

Subnet name

Create a tag with a key of 'Name' and a value that you specify.

DC-Public-Subnet

The name can be up to 256 characters long.

Availability Zone [Info](#)

Choose the zone in which your subnet will reside, or let Amazon choose one for you.

United States (N. Virginia) / use1-az4 (us-east-1a)

IPv4 VPC CIDR block [Info](#)

Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

192.168.0.0/16

IPv4 subnet CIDR block

192.168.0.0/24 256 IPs

Private Subnet:

Subnet 2 of 2

Subnet name

Create a tag with a key of 'Name' and a value that you specify.

DC-Private-Subnet

The name can be up to 256 characters long.

Availability Zone [Info](#)

Choose the zone in which your subnet will reside, or let Amazon choose one for you.

No preference

IPv4 VPC CIDR block [Info](#)

Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

192.168.0.0/16

IPv4 subnet CIDR block

192.168.100.0/24 256 IPs

Create separate **Route Tables** for the public and private subnets.

Create route table [Info](#)

A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.

Route table settings

Name - optional

Create a tag with a key of 'Name' and a value that you specify.

DC-Public-RT

VPC

The VPC to use for this route table.

vpc-0fc94dc2cfdab75d0 (DC-VPC)

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Q Name X

Value - optional

Q DC-Public-RT X

Remove

Add new tag

You can add 49 more tags.

Cancel

Create route table

Associate the public subnet with a route table containing a default route (0.0.0.0/0) to the Internet Gateway.

VPC > Route tables > rtb-0e18bce0a117ede8d > Edit routes

Edit routes

Destination	Target	Status	Propagated	Route Origin
192.168.0.0/16	local	Active	No	CreateRouteTable
0.0.0.0/0	Internet Gateway	-	No	CreateRoute

[Add route](#) [Cancel](#) [Preview](#) [Save changes](#)

Associate with the Public subnet

Edit subnet associations

Change which subnets are associated with this route table.

Available subnets (1/2)

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR	Route table ID
DC-Private-Subnet	subnet-0267ef829dc2bc474	192.168.100.0/24	-	Main (rtb-0f50a8df278f1c8f2)
DC-Public-Subnet	subnet-0321deeb2a4666005	192.168.0.0/24	-	Main (rtb-0f50a8df278f1c8f2)

Selected subnets

subnet-0321deeb2a4666005 / DC-Public-Subnet

[Cancel](#) [Save associations](#)

Create the Private Route Table

Create route table

A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.

Route table settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

DC-Private-RT

VPC
The VPC to use for this route table.

vpc-0fc94dc2cfdab75d0 (DC-VPC)

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional
Name	DC-Private-RT

[Add new tag](#) [Remove](#)

You can add 49 more tags.

[Cancel](#) [Create route table](#)

Associate to the private subnet

VPC > Route tables > rtb-0346d680e239eb22d > Edit subnet associations

Edit subnet associations

Change which subnets are associated with this route table.

Available subnets (1/2)

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR	Route table ID
DC-Private-Subnet	subnet-0267ef829dc2bc474	192.168.100.0/24	-	Main (rtb-0f50a8df278f1c8f2)
DC-Public-Subnet	subnet-0321deeb2a4666005	192.168.0.0/24	-	rtb-0e18bce0a117ede8d / DC-Public-RT

Selected subnets

subnet-0267ef829dc2bc474 / DC-Private-Subnet

[Cancel](#) [Save associations](#)

Choose AMI as Amazon Linux.

Network Settings

Launch another EC2 instance named **EC2-B** inside the private subnet. Disable **Auto-assign Public IP** and configure the Security Group to allow **SSH** from the data center network (192.168.0.0/16) and **ICMP** from the AWS VPC for connectivity testing.

Network Setting

Inbound Security Group Rules

▼ Security group rule 1 (TCP, 22, 192.168.0.0/16)

Remove

Type | Info

Protocol | Info

Port range | Info

ssh

TCP

22

Source type | Info

Source | Info

Description - optional | Info

Custom

Q Add CIDR, prefix list or security group

192.168.0.0/16 X

e.g. SSH for admin desktop

Return to the **Mumbai** region and create a **Virtual Private Gateway (VGW)**. After the gateway is created, attach it to **AWS-VPC**, allowing the VPC to terminate Site-to-Site VPN connections.

Attach it to the VPC

You successfully created vgw-07b68dc79af90ab2e / aws-network-vgw.

Virtual private gateways (1/1) [Info](#)

Find resource by attribute or tag

Name	Virtual private gateway ID	State	VPC attachment state
aws-network-vgw	vgw-07b68dc79af90ab2e	Available	Detached

Actions

- Attach to VPC
- Detach from VPC
- Manage tags
- Delete virtual private gateway

Attach to VPC [Info](#)

Attach the virtual private gateway to this VPC.

Details

Virtual private gateway ID

vgw-07b68dc79af90ab2e

Available VPCs

Attach the virtual private gateway to this VPC.

vpc-03b0d69e546a3e416 / AWS-VPC

[Cancel](#) [Attach to VPC](#)

Create a **Customer Gateway** representing the on-premises VPN device. Use the **Public IP Address** of the **VPN-Server EC2** instance as the Customer Gateway IP, since this server acts as the simulated VPN router.

Create customer gateway [Info](#)

A customer gateway is a resource that you create in AWS that represents the customer gateway device in your on-premises network.

Details

Name tag - optional

Creates a tag with a key of 'Name' and a value that you specify.

dc-network-cgw

Value must be 256 characters or less in length.

BGP ASN [Info](#)

The ASN of your customer gateway device.

65000

Value must be in 1 - 4294967294 range.

IP address [Info](#)

Specify the IP address for your customer gateway device's external interface.

54.175.250.240

Certificate ARN - optional

The ARN of a private certificate provisioned in AWS Certificate Manager (ACM).

Select certificate ARN

Device - optional

Enter a name for the customer gateway device.

Enter device name

Create a **Site-to-Site VPN Connection** by selecting the Virtual Private Gateway as the AWS endpoint and the Customer Gateway as the remote endpoint.

Create VPN connection [Info](#)

Select the resources and additional configuration options that you want to use for the site-to-site VPN connection.

Details

Name tag - optional

Creates a tag with a key of 'Name' and a value that you specify.

aws-dc-vpn

Value must be 256 characters or less in length.

Target gateway type [Info](#)

☒ Virtual private gateway

☐ Transit gateway

☐ Site-to-Site VPN Concentrator

☐ Not associated

Virtual private gateway

vgw-07b68dc79af90ab2e

Customer gateway [Info](#)

☒ Existing

☐ New

Customer gateway ID

cgw-04190ae997d9acee8

Choose **Static Routing** and specify the on-premises network CIDR **192.168.0.0/16**.

Routing options | Info

☐ Dynamic (requires BGP)

☒ **Static**

Static IP prefixes | Info

Q Add static IP prefix

192.168.0.0/16 X

Pre-shared key storage | Info

☒ Standard

☐ Secrets Manager

Local IPv4 network CIDR - optional

The IPv4 CIDR range on the customer gateway (on-premises) side that is allowed to communicate over the VPN tunnels. The default is 0.0.0.0/0.

Q 0.0.0.0/0

Remote IPv4 network CIDR - optional

The IPv4 CIDR range on the AWS side that is allowed to communicate over the VPN tunnels. The default is 0.0.0.0/0.

Q 0.0.0.0/0

Wait until the VPN connection is created, then download the **VPN Configuration File**. Select the **OpenSwan** configuration format, as the VPN server will use LibreSwan to establish the IPsec tunnel.

VPN connections (1/1) | Info

Find resource by attribute or tag

Name	VPN ID
aws-dc-vpn	vpn-0305d5dfd084fe9c

VPN connection vpn-0305d5dfd084fe9c

Details | Tunnel details | Status

Details

VPN ID
vpn-0305d5dfd084fe9c

Transit gateway
-

VPC
vpc-03b0d69e546a3e416

Download configuration

Download a sample configuration based on your customer gateway. Note that this is a sample only, and that it will require modification for using Advanced Algorithms, Certificates, and/IPv6.

Vendor
The manufacturer of the customer gateway device (for example, Cisco Systems, Inc.).

Openswan

Platform
The class of the customer gateway device (for example, J-Series).

Openswan

Software
The operating system running on the customer gateway device (for example, ScreenOS).

Openswan 2.6.38+

IKE version
The IKE version you are using for your VPN connection.

ikev1

Include sample type - optional

☐ Enable

Sample type
The default sample type compatibility mode includes all options. The recommended mode restricts options to only the most secure settings (IKEv2, etc.).

Select sample type

Cancel Download

Customer gateway
cgw-04190ae997d9acee8

Category
VPN

Authentication
Pre-shared key

Configure the VPN Server

Connect to the **VPN-Server** using SSH or EC2 Instance Connect and install the **LibreSwan** VPN software. This software will establish the IPsec VPN tunnel between the simulated data center and AWS.

```
sudo dnf update -y
```

```
sudo dnf install libreswan -y
```

Now the file that you have downloaded

```
-----
IPSEC Tunnel #1
-----

This configuration assumes that you already have a default openswan installation in place on the Amazon Linux operating system (but may also work with other distros as well)

1) Open /etc/sysctl.conf and ensure that its values match the following:
net.ipv4.ip_forward = 1
net.ipv4.conf.default.rp_filter = 0
net.ipv4.conf.default.accept_source_route = 0
```

Configure the required **sysctl** parameters to enable packet forwarding and IPsec functionality.

```
[ec2-user@ip-192-168-0-62 ~]$ sudo nano /etc/sysctl.conf
[ec2-user@ip-192-168-0-62 ~]$ cat /etc/sysctl.conf
# sysctl settings are defined through files in
# /usr/lib/sysctl.d/, /run/sysctl.d/, and /etc/sysctl.d/.
#
# Vendors settings live in /usr/lib/sysctl.d/.
# To override a whole file, create a new file with the same in
# /etc/sysctl.d/ and put new settings there. To override
# only specific settings, add a file with a lexically later
# name in /etc/sysctl.d/ and put new settings there.
#
# For more information, see sysctl.conf(5) and sysctl.d(5).

net.ipv4.ip_forward = 1
net.ipv4.conf.default.rp_filter = 0
net.ipv4.conf.default.accept_source_route = 0
```

Now run the next command to apply the changes

```
sudo sysctl -p
```

```
[ec2-user@ip-192-168-0-62 ~]$ sudo sysctl -p
net.ipv4.ip_forward = 1
net.ipv4.conf.default.rp_filter = 0
net.ipv4.conf.default.accept_source_route = 0
[ec2-user@ip-192-168-0-62 ~]$
```

Open `/etc/ipsec.conf` and look for the line below. Ensure that the `#` in front of the line has been removed, then save and exit the file.

```
#include /etc/ipsec.d/*.conf
```

```
cat /etc/ipsec.conf
```

```
config setup
# If logfile= is unset, syslog is used to send log messages too.
# Note that on busy VPN servers, the amount of logging can trigger
# syslogd (or journald) to rate limit messages.
#logfile=/var/log/pluto.log
#
# Debugging should only be used to find bugs, not configuration issues!
# "base" regular debug, "tmi" is excessive (!) and "private" will log
# sensitive key material (not available in FIPS mode). The "cpu-usage"
# value logs timing information and should not be used with other
# debug options as it will defeat getting accurate timing information.
# Default is "none"
#plutodebug="base"
#plutodebug="tmi"
#plutodebug="none"
#
# Some machines use a DNS resolver on localhost with broken DNSSEC
# support. This can be tested using the command:
# dig +dnssec DNSNameOfRemoteServer
# If that fails but omitting '+dnssec' works, the system's resolver is
# broken and you might need to disable DNSSEC.
# dnssec-enable=no
#
# To enable IKE and IPsec over TCP for VPN server. Requires at least
# Linux 5.7 kernel or a kernel with TCP backport (like RHEL8 4.18.0-291)
# listen-tcp=yes
# To enable IKE and IPsec over TCP for VPN client, also specify
# tcp-remote-port=4500 in the client's conn section.

# if it exists, include system wide crypto-policy defaults
include /etc/crypto-policies/back-ends/libreswan.config

# It is best to add your IPsec connections as separate files
# in /etc/ipsec.d/
include /etc/ipsec.d/*.conf
[ec2-user@ip-192-168-0-62 ~]$
```

Create the required **LibreSwan configuration files** (`aws.conf` and `aws.secrets`) using the information from the downloaded AWS VPN configuration file. Update the configuration to match the local and remote subnet CIDR ranges and apply the recommended encryption settings.

```
sudo nano /etc/ipsec.d/aws.conf
```

Make the changes according to the instruction given in the image

```

conn Tunnell
    authby=secret
    auto=start
    left=%defaultroute
    leftid=98.80.190.150
    right=13.127.176.202
    type=tunnel
    ikelifetime=8h
    keylife=1h
    phase2alg=aes128-sha256
    ike=aes128-sha256;modp2048
    auth=esp
    keyingtries=%forever
    keyexchange=ike
    leftsubnet=192.168.0.0/16
    rightsubnet=10.0.0.0/16
    dpddelay=10
    dpdtimeout=30
    dpdaction=restart_by_peer

```

← Should be VPN Server Public IP
 ← Should be one of the two AWS VGW Tunnels' Public IP
 ← Update to these values
 ← Remove
 ← Update to DC-VPC network CIDR
 ← Update to AWS-VPC network CIDR

```

[ec2-user@ip-192-168-0-62 ~]$ sudo cat /etc/ipsec.d/aws.conf
conn Tunnell
    authby=secret
    auto=start
    left=%defaultroute
    leftid=54.175.250.240
    right=13.205.229.115
    type=tunnel
    ikelifetime=8h
    keylife=1h
    phase2alg=aes128-sha256
    ike=aes128-sha256;modp2048
    keyingtries=%forever
    keyexchange=ike
    leftsubnet=192.168.0.0/16
    rightsubnet=10.0.0.0/16
    dpddelay=10
    dpdtimeout=30
    dpdaction=restart_by_peer
[ec2-user@ip-192-168-0-62 ~]$

```

Now for the aws.secret file follow the step 5 from the file copy the below line

```

5) Create a new file at /etc/ipsec.d/aws.secrets if it doesn't already exist, and append this line to the file (be mindful of the spacing!):
54.175.250.240 13.205.229.115: PSK "YBg1T_lpbI8hiZeJ7LTjb2y2LR81zk_B"

```

Run the command

```
sudo nano /etc/ipsec.d/aws.secrets
```

```

[ec2-user@ip-192-168-0-62 ~]$ sudo nano /etc/ipsec.d/aws.secrets
[ec2-user@ip-192-168-0-62 ~]$

```

It should look like

```

[ec2-user@ip-192-168-0-62 ~]$ sudo cat /etc/ipsec.d/aws.secrets
54.175.250.240 13.205.229.115: PSK "YBg1T_lpbI8hiZeJ7LTjb2y2LR81zk_B"
[ec2-user@ip-192-168-0-62 ~]$

```

Start the **IPsec service** and verify that the VPN tunnel begins negotiating with AWS.

Although the service may start successfully, connectivity testing is required to confirm that the VPN tunnel is fully operational.

Run the command

```
sudo systemctl start ipsec.service
```

then check the status

```
sudo systemctl status ipsec.service
```

the status should be running

```
[ec2-user@ip-192-168-0-62 ~]$ sudo systemctl start ipsec.service
[ec2-user@ip-192-168-0-62 ~]$ sudo systemctl status ipsec.service
• ipsec.service - Internet Key Exchange (IKE) Protocol Daemon for IPsec
   Loaded: loaded (/usr/lib/systemd/system/ipsec.service; disabled; preset: disabled)
   Active: active (running) since Wed 2026-07-01 10:16:37 UTC; 5s ago
     Docs: man:ipsec(8),
           man:pluto(8),
           man:ipsec.conf(5).
   Process: 29156 ExecStartPre=/usr/libexec/ipsec/addconn --config /etc/ipsec.conf --checkconfig (code=exited, status=0/SUCCESS)
   Process: 29157 ExecStartPre=/usr/libexec/ipsec/stackmanager start (code=exited, status=0/SUCCESS)
   Process: 29432 ExecStartPre=/usr/sbin/ipsec --checknss (code=exited, status=0/SUCCESS)
   Process: 29437 ExecStartPre=/usr/sbin/ipsec --checknlog (code=exited, status=0/SUCCESS)
 Main PID: 29448 (pluto)
   Status: "Startup completed."
    Tasks: 3 (limit: 1059)
   Memory: 8.6M
     CPU: 608ms
   CGroup: /system.slice/ipsec.service
           └─29448 /usr/libexec/ipsec/pluto --leak-detective --config /etc/ipsec.conf --nofork
```

To check its working, let's ping the private IP address of the instance created in Mumbai region

```
[ec2-user@ip-192-168-0-62 ~]$ ping 10.0.0.131
PING 10.0.0.131 (10.0.0.131) 56(84) bytes of data.
```

It's not working because access through the Virtual private Gateway

Return to the **AWS Private Route Table** in the Mumbai region and add a new route for **192.168.0.0/16**. Select the **Virtual Private Gateway** as the target so that traffic destined for the on-premises network is sent through the VPN tunnel.

The screenshot shows the AWS Management Console interface for editing a route table. The breadcrumb navigation at the top indicates the path: VPC > Route tables > rtb-0d8ca228a9d273cc8 > Edit routes. The main section is titled 'Edit routes'. It contains a table with columns: Destination, Target, Status, Propagated, and Route Origin. The first row shows an existing route for destination 10.0.0.0/16 with target 'local', status 'Active', and origin 'CreateRouteTable'. A second row is highlighted with a red box, showing a new route being added: destination '192.168.0.0/16', target 'Virtual Private Gateway', status '-', and origin 'CreateRoute'. Below the table, there is an 'Add route' button. At the bottom right, there are 'Cancel', 'Preview', and 'Save changes' buttons.

Now it starting working.

```
[ec2-user@ip-192-168-0-62 ~]$ ping 10.0.0.131
PING 10.0.0.131 (10.0.0.131) 56(84) bytes of data.
 64 bytes from 10.0.0.131: icmp_seq=176 ttl=127 time=202 ms
 64 bytes from 10.0.0.131: icmp_seq=177 ttl=127 time=202 ms
 64 bytes from 10.0.0.131: icmp_seq=178 ttl=127 time=202 ms
 64 bytes from 10.0.0.131: icmp_seq=179 ttl=127 time=202 ms
 64 bytes from 10.0.0.131: icmp_seq=180 ttl=127 time=202 ms
 64 bytes from 10.0.0.131: icmp_seq=181 ttl=127 time=202 ms
 64 bytes from 10.0.0.131: icmp_seq=182 ttl=127 time=202 ms
 64 bytes from 10.0.0.131: icmp_seq=183 ttl=127 time=202 ms
 64 bytes from 10.0.0.131: icmp_seq=184 ttl=127 time=202 ms
 64 bytes from 10.0.0.131: icmp_seq=185 ttl=127 time=202 ms
 64 bytes from 10.0.0.131: icmp_seq=186 ttl=127 time=202 ms
```

Now you can see one of the tunnel is up

VPN connections (1/1) Info

Find resource by attribute or tag

Name	VPN ID	State	Virtual private gateway	Transit gateway	VPN Concentra
aws-dc-vpn	vpn-0305d5dfdf084fe9c	Available	vgw-07b68dc79af90ab2e	-	-

VPN connection vpn-0305d5dfdf084fe9c / aws-dc-vpn

Details | **Tunnel details** | Static routes | Tags

This VPN connection is not using both tunnels. This mode of operation is not highly available and we strongly recommend you configure your second tunnel.

Tunnel state

Tunnel number	Outside IP address	Inside IPv4 CIDR	Inside IPv6 CIDR	Status	Provisioning status	Last status change
Tunnel 1	13.205.229.115	169.254.60.180/30	-	Up	Available	July 1, 2026, 15:46:58 (UTC+05:30)
Tunnel 2	65.2.229.100	169.254.253.140/30	-	Down	Available	July 1, 2026, 14:52:27 (UTC+05:30)

On the Data Center side, disable **Source/Destination Check** on the **VPN-Server EC2** instance. This allows the instance to forward traffic between the private subnet and the AWS VPC, enabling it to function as a router.

Go to VPN-Server > Networking > Change Source/Destination Check.

Instances (1/2) Info

Connect | Instance state | **Actions** | Launch instances

Find Instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance state
VPN-Server	i-043ccb96a24f72b34	Running
EC2-B	i-0fede835d39ad1fae	Running

Attach network interface
Detach network interface
Manage IP addresses
Disassociate Elastic IP address
Change source/destination check
Manage ENA express
Manage ENA queues
Manage bandwidth

Instance diagnostics
Instance settings
Networking
Security
Image and templates
Storage
Monitor and troubleshoot

Check the stop and save

Change Source / destination check

The source / destination check ensures that the instance is the source or destination of all the traffic it sends and receives. Each EC2 instance performs source and destination checks by default. [Learn more](#)

Instance ID
i-043ccb96a24f72b34 (VPN-Server)

Network interface
eni-0b55aa22637db737d

Source / destination checking
Stop to allow your instance to send and receive traffic when the source or destination is not itself.

☒ Stop

Cancel Save

Public IPv4 address
54.175.250.240 | [open address](#)

Private IPv4 addresses
192.168.0.62

Edit the **Private Route Table** of the Data Center VPC and add a route for the AWS network (**10.0.0.0/16**). Select the **VPN-Server EC2 instance** as the route target so that all AWS-bound traffic is forwarded through the VPN server.

The screenshot shows the 'Edit routes' interface in the AWS console. The 'Destination' field contains '192.168.0.0/16'. The 'Target' dropdown is set to 'local'. Below this, a search for '10.0.0.0/16' is shown, with a list of results including 'I-043ccb96a24f72b34 (VPN-Server)'. The 'Add route' button is highlighted with a red box. The 'Status' column shows 'Active' for the local route and 'No' for the new route. The 'Route Origin' column shows 'CreateRouteTable' for the local route and 'CreateRoute' for the new route.

Connect to the **VPN-Server** and use SSH to access **EC2-B** in the private subnet using its **Private IP Address**. Since both instances are in the same VPC, the VPN server can reach EC2-B through the private network.

Connect to the VPN-Server instance and create a file key.pem file

```
[ec2-user@ip-192-168-0-62 ~]$ nano key.pem
[ec2-user@ip-192-168-0-62 ~]$
```

And copy the content of the EC2-B's Key and paste there in the key.pem file.

The screenshot shows the nano text editor with the content of the EC2-B's Key file being pasted into the key.pem file. The content is a long string of characters, including '-----BEGIN PRIVATE KEY-----' and '-----END PRIVATE KEY-----'.

Save the file

Now change the permission of the key

Run the command

```
chmod 400 key.pem
```

```
[ec2-user@ip-192-168-0-62 ~]$ nano key.pem
[ec2-user@ip-192-168-0-62 ~]$ chmod 400 key.pem
[ec2-user@ip-192-168-0-62 ~]$
```

Now connect to the EC2-B instance using ssh

```
ssh -i key.pem <private_IP_address>
```

```
[ec2-user@ip-192-168-0-62 ~]$ ssh -i key.pem 192.168.100.147
The authenticity of host '192.168.100.147 (192.168.100.147)' can't be established.
ED25519 key fingerprint is SHA256:yve6mTszswDJS/j8sVmNm0RGagpP75XMBM65NMtAFz4.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.100.147' (ED25519) to the list of known hosts.
```

Ping the EC2-A through private IP address from this instance

We are able to connect it to the EC2-A instance that was in mumbai region.

```
[ec2-user@ip-192-168-100-147 ~]$ ping 10.0.0.131
PING 10.0.0.131 (10.0.0.131) 56(84) bytes of data:
64 bytes from 10.0.0.131: icmp_seq=1 ttl=126 time=185 ms
64 bytes from 10.0.0.131: icmp_seq=2 ttl=126 time=185 ms
64 bytes from 10.0.0.131: icmp_seq=3 ttl=126 time=185 ms
64 bytes from 10.0.0.131: icmp_seq=4 ttl=126 time=185 ms
```